IJARSCT
International Journal of Advanced Research in Science, Communication and Technology
International Open-Access, Double-Blind, Peer-Reviewed, Refereed, Multidisciplinary Online Journal
ISSN: 2581-9429
Volume 5, Issue 4, November 2025
Impact Factor: 7.67

Conceptual Design of an LLM-Based Tech Product Recommendation System Using LangChain, LangGraph, Firecrawl, and n8n with RAG, Fine-Tuning, Prompt Engineering, and KNN with Cosine Similarity

Mrs. Abha Pathak, Mrs. Tejaswini Mali, Mr. Sanket Rathod
Mr. Niraj Rane, Ms. Aditi Rakh, Ms. Swapnali Pimpare
Department of Computer Engineering
Dr. D. Y. Patil College of Engineering and Innovation, Varale, Talegaon, Pune, MH, India
abhanayak@gmail.com, tejaswinipatil403@gmail.com, sanketnrathod.sr@gmail.com
nirajrane325@gmail.com, aditirakh05@gmail.com, pimpareswapnali@gmail.com

Abstract: *Choosing the right technology product has become increasingly difficult for consumers due to limited technical knowledge, rapidly evolving specifications, and the overwhelming number of available options. Traditional recommendation systems rely on static filters or keyword-based searches, often producing incomplete or context-insensitive results. This paper proposes a conceptual design for an AI-driven recommendation framework that leverages Large Language Models (LLMs) to deliver accurate, explainable, and personalized product suggestions. The system integrates LangChain and LangGraph to manage reasoning, tool orchestration, and multi-step control flow, while product similarity is computed using K-Nearest Neighbors (KNN) with cosine similarity. To ensure factual grounding and reduce hallucination, the design incorporates Retrieval-Augmented Generation (RAG), complemented by fine-tuning and prompt engineering for domain-specific alignment. A continuously updated product knowledge base, maintained through automated web scraping using Firecrawl and workflow synchronization via n8n, supports real-time data accuracy. The proposed framework enables natural-language interaction and aims to provide reliable recommendations for devices such as smartphones, laptops, and wearables, offering a scalable and modular foundation for next-generation tech product advisory systems.*

Keywords: Artificial Intelligence, automation, consumer electronics, cosine similarity, fine-tuning, KNN, LangChain, LangGraph, large language models, prompt engineering, RAG, recommendation system.

I. INTRODUCTION

Large Language Models (LLMs) have significantly advanced modern Artificial Intelligence (AI), especially in information filtering and recommender systems (RS). Traditional RS methods enhance user experience and engagement but often rely on static filters or interaction histories, making them less effective for complex domains such as technology products. For consumers, selecting the right device has become increasingly difficult due to limited technical knowledge, rapidly evolving specifications, and the overwhelming variety of available options. These challenges make it harder for users to compare products objectively or interpret detailed specifications without expert guidance. Although LLMs provide strong reasoning and conversational abilities, deploying them for specialized, domain-specific recommendation tasks requires careful optimization and robust system design.

IJARSCT
International Journal of Advanced Research in Science, Communication and Technology
International Open-Access, Double-Blind, Peer-Reviewed, Refereed, Multidisciplinary Online Journal
ISSN: 2581-9429
Volume 5, Issue 4, November 2025
Impact Factor: 7.67

A. Problem Statement

Conventional recommendation approaches—such as collaborative filtering and content-based methods—face well-known limitations, including cold-start issues, data sparsity, and scalability constraints. While LLMs offer an opportunity for more interactive and context-aware recommendations, general-purpose models lack the fine-grained domain knowledge needed for accurate tech product suggestions. A major challenge is LLM hallucination, where models produce fluent but incorrect information, making them unreliable for factual tasks like product specification retrieval.

Building an LLM-based recommendation pipeline also introduces architectural complexity. Integrating models with multiple tools, databases, and external sources can lead to an N×M integration problem, where every new component requires custom connections. The Model Context Protocol (MCP) provides a standardized interface for connecting LLMs to external tools and services, reducing the need for custom integrations and mitigating the N×M scaling challenge. Many existing solutions do not combine LLM alignment techniques with automated, continuously updated data management frameworks.

B. Motivation

The motivation for this conceptual design also arises from the growing difficulty consumers face when selecting technology products. Most users have limited technical knowledge and often struggle to understand specifications such as processors, sensors, camera architectures, battery technologies, and software features. Additionally, the huge number of available options—with frequent new releases—creates confusion and makes it difficult for users to compare devices objectively. Therefore, there is a need for a system that can simplify decision-making by providing clear, factual, and context-aware recommendations in natural language.

- From a technical standpoint, two major requirements must be addressed:
- achieving high factual accuracy in domain-specific recommendations, and
- enabling scalable, automated orchestration of LLM interactions with external tools and data sources.

Retrieval-Augmented Generation (RAG) helps address hallucination by grounding responses in verified, up-to-date product data. Fine-tuning further adapts the LLM to domain vocabulary, while prompt engineering ensures structured and interpretable outputs. To maintain data relevance, the design integrates orchestration frameworks such as LangChain and LangGraph along with automated data pipelines using Firecrawl for real-time scraping and n8n for scheduling, synchronization, and manual verification before database integration.

C. Research Gaps

Existing research often examines LLM optimization or traditional recommendation algorithms separately. There is a lack of unified designs that integrate:

- RAG, fine-tuning, and prompt engineering,
- Vector-based recommendation (KNN with cosine similarity),
- Orchestration frameworks (LangChain and LangGraph), and
- Automated data pipelines (Firecrawl and n8n).

A cohesive architecture that combines these components for semantic search and conversational recommendations remains unexplored.

D. Proposed Conceptual Solution

This paper proposes a modular, layered architectural design for an LLM-based tech product recommendation system. The system uses LangChain for orchestration, LangGraph for graph-based control flow, RAG for factual grounding, Firecrawl and n8n for data management, and KNN with cosine similarity for similarity ranking. Fine-tuning and prompt engineering strengthen domain alignment and improve the clarity and structure of generated responses.

IJARSCT
International Journal of Advanced Research in Science, Communication and Technology
International Open-Access, Double-Blind, Peer-Reviewed, Refereed, Multidisciplinary Online Journal
ISSN: 2581-9429
Volume 5, Issue 4, November 2025
Impact Factor: 7.67

V. Anticipated Contributions

The major conceptual contributions include:

- A unified architectural paradigm integrating LLM orchestration, automated data ingestion, and workflow management.
- Improved recommendation quality through the complementary use of RAG, fine-tuning, and prompt engineering.
- A continuous data synchronization pipeline with human-verified updates using Firecrawl and n8n.
- An adaptive and interpretable reasoning framework enabled by LangGraph's graph-based control flow.

II. LITERATURE SURVEY

This section reviews the established foundational concepts and recent technical advancements underpinning the proposed LLM-based tech product recommendation system, covering recommendation methodologies, LLM optimization, and architectural frameworks.

A. Tech Product Recommendation Systems

Recommender systems (RS) serve as critical information filtering tools that predict user interest in items such as e-commerce products. Traditional methods, including collaborative filtering and content-based filtering, face inherent limitations such as the cold start problem, where new items or users lack interaction history; data sparsity, resulting from incomplete interaction matrices; and scalability issues. The shift toward modern AI-based approaches addresses these gaps, but the complex, unstructured nature of tech product specifications requires adaptive architectures capable of utilizing rich, domain-specific knowledge.

B. Large Language Models in Recommendation Systems

The Transformer architecture and resulting Large Language Models (LLMs) have enabled new capabilities in RS, particularly conversational recommender systems (CRSs) that allow for interactive, context-aware dialogue. However, integrating general-purpose LLMs into specialized domains like tech products introduces challenges, primarily due to their knowledge cut-off (lack of information on new items) and factual inaccuracy or hallucination. This risk of generating plausible but false product specifications renders ungrounded LLMs unsuitable for tasks requiring high factual precision.

C. LangChain and LangGraph for LLM Orchestration

Effective deployment of LLMs requires robust orchestration frameworks to manage their interaction with external tools, data sources, and proprietary algorithms. LangChain acts as the core orchestration framework, facilitating prompt construction, retrieval integration, and general model-data interaction. This approach helps mitigate the complex "N×M integration problem" that arises when connecting multiple LLMs (N) with various external tools (M). LangGraph extends this capability by enabling graph-based control flow, which is essential for defining modular, scalable, and stateful reasoning paths in complex, multi-actor AI applications. Additionally, the Model Context Protocol (MCP) introduces a standardized mechanism for tool discovery and interoperability, enabling seamless integration of external services into LLM-driven workflows.

D. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is a critical optimization technique that addresses LLM hallucination and knowledge cut-off by dynamically grounding the model's response in external, factual data. The RAG pipeline involves indexing the knowledge base by converting text chunks into vector embeddings using an embedding model. The Retriever then searches this vector database (e.g., FAISS) for the top-k semantically relevant text chunks based on cosine similarity with the user query. These retrieved chunks are inserted into the LLM's prompt, allowing the

IJARSCT
International Journal of Advanced Research in Science, Communication and Technology
International Open-Access, Double-Blind, Peer-Reviewed, Refereed, Multidisciplinary Online Journal
ISSN: 2581-9429 Volume 5, Issue 4, November 2025 Impact Factor: 7.67

Generator to produce a factual response that is grounded in the external context. RAG significantly enhances factual relevance and improves the explainability of the output by enabling the system to cite sources.

E. Fine-Tuning LLMs

Fine-Tuning is a specialized model optimization technique that complements RAG by adjusting the LLM's internal parameters to better understand domain-specific language and task requirements. This process involves training a generic pre-trained model on a smaller, labeled, domain-specific dataset, adapting its internal representations to nuances specific to the tech product vocabulary, thereby enhancing precision. Techniques like QLoRA (Quantized Low-Rank Adaptation) increase efficiency by reducing the computational memory required during the fine-tuning process. However, fine-tuning requires substantial labeled data and is computationally expensive, often necessitating retraining to incorporate new knowledge, unlike RAG.

F. Prompt Engineering Techniques

Prompt Engineering is the strategic design of text inputs to guide the LLM toward desired outputs without modifying the model's weights. It is essential for constraining the LLM to deliver recommendations in a structured, consistent format, which is crucial when integrating LLM output into software pipelines expecting precise schema (e.g., JSON). Advanced methods, such as Chain-of-Thought (CoT) Prompting, instruct the LLM to break down complex problems into intermediate steps, significantly enhancing the model's reasoning capabilities before generating the final answer. Prompt engineering offers a flexible and cost-effective method for quickly optimizing LLM behavior.

G. Knowledge-Based Recommendation Systems and Data Automation

Knowledge-Based Recommendation Systems (KBRS) utilize structured knowledge about items, often employing graphs, to provide personalized and explainable recommendations. Crucially, maintaining the freshness and accuracy of this product knowledge base is paramount in a dynamic domain like technology. The proposed architecture employs dedicated tools for continuous data synchronization. Firecrawl is integrated for automated web scraping of current tech product data, enabling the system to continuously update the knowledge base with the latest specifications and features. n8n is leveraged as the workflow automation layer to manage scheduled data ingestion via Firecrawl, orchestrate subsequent data preprocessing steps, and trigger the necessary database re-indexing, ensuring the RAG pipeline always accesses timely and proprietary information.

H. Nearest Neighbors (KNN) and Cosine Similarity

To quantify product similarity based on vectorized representations, the system incorporates the classic machine learning algorithm K-Nearest Neighbors (KNN). Recommendation features, derived from processed product data and user preferences, are converted into high-dimensional vector representations (embeddings). Cosine Similarity is the metric used to measure the angle between these vectors, quantifying their semantic proximity in the embedding space. This combination efficiently identifies the most relevant and semantically similar items for recommendation within the overall framework.

III. SYSTEM ARCHITECTURE AND DESIGN

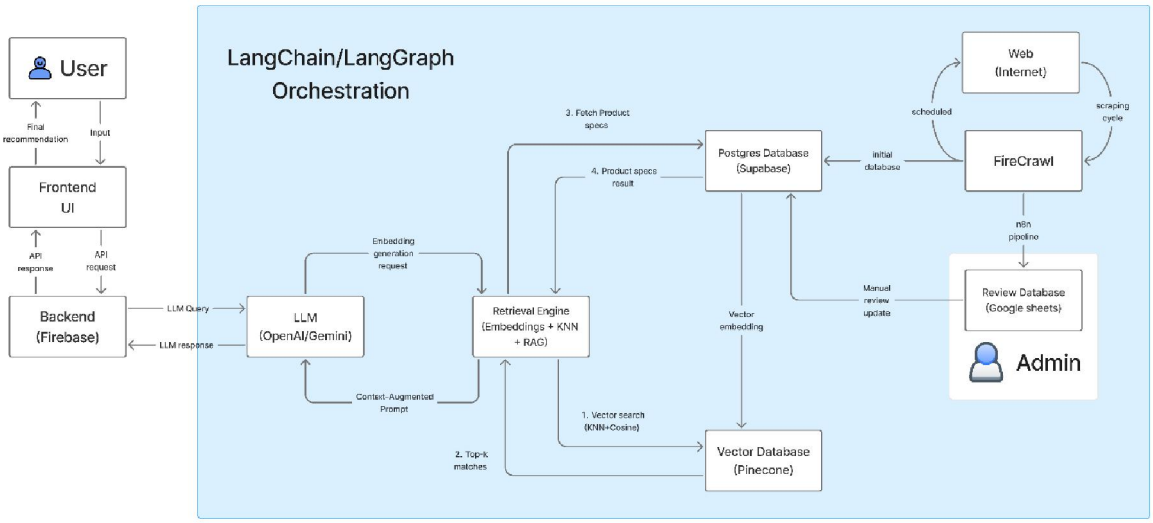
The conceptual design for the LLM-Based Tech Product Recommendation System employs a modular, layered architecture to manage continuous data synchronization, context-aware reasoning, and algorithmic recommendation. This structure systematically integrates the Large Language Model (LLM) core with specialized orchestration frameworks and external tools to achieve accurate and explainable recommendations in the domain of technology products.

A. Overall System Overview

The architecture is organized into six functional layers, each responsible for a specific aspect of the recommendation pipeline:

IJARSCT
International Journal of Advanced Research in Science, Communication and Technology
International Open-Access, Double-Blind, Peer-Reviewed, Refereed, Multidisciplinary Online Journal
ISSN: 2581-9429
Volume 5, Issue 4, November 2025
Impact Factor: 7.67

- Data Ingestion and Automation Layer – Uses Firecrawl and n8n to scrape, collect, normalize, and verify product data.
- Knowledge Base / Vector Store Layer – Stores domain-specific product embeddings and text chunks for semantic retrieval.
- Language Understanding Layer – Managed through LangChain and LangGraph, enabling structured reasoning and tool orchestration.
- Retrieval and Computation Layer – Executes Retrieval-Augmented Generation (RAG) and K-Nearest Neighbors (KNN) similarity matching.
- LLM Core Layer – The central reasoning engine, enhanced via Fine-Tuning for domain alignment.
- Response Generation Layer – Uses Prompt Engineering to transform retrieved insights into structured natural-language recommendations.



The layered design ensures that user queries are progressively interpreted, grounded with factual data, ranked algorithmically, and synthesized into reliable, human-interpretable outputs.

B. Component Breakdown

The efficacy of this architecture rests upon the defined roles of its primary components. The LLM Core serves as the central reasoning engine, providing complex language generation capabilities. LangChain is utilized as the primary LLM orchestration layer for prompt construction and general model-tool interaction. LangGraph extends this by establishing graph-based control flow, necessary for complex, stateful, and multi-turn recommendation paths. The Model Context Protocol (MCP) may also be incorporated to expose external tools and utilities through a unified protocol, simplifying multi-component coordination within the architecture.Firecrawl automates data ingestion by scraping and normalizing tech product data. n8n manages the Automation Layer, scheduling periodic data updates and synchronizing workflow tasks. The Recommendation Engine combines RAG, Fine-Tuning, Prompt Engineering, and KNN with Cosine Similarity to optimize recommendation relevance.

IJARSCT
International Journal of Advanced Research in Science, Communication and Technology
International Open-Access, Double-Blind, Peer-Reviewed, Refereed, Multidisciplinary Online Journal
ISSN: 2581-9429 Volume 5, Issue 4, November 2025 Impact Factor: 7.67

C. Data Acquisition and Processing with Firecrawl

To maintain an accurate, up-to-date knowledge base, the system uses an automated ingestion pipeline supported by a Human-in-the-Loop (HITL) verification stage. Firecrawl scrapes current tech product specifications and forwards the extracted content to n8n, which performs preliminary preprocessing and stores it in a temporary Review Database.

Before entering the main knowledge base, all entries undergo HITL validation (see Section VI), where human reviewers confirm factual correctness and consistency. Once approved, n8n forwards the validated data to the automated processing pipeline, which includes:

- normalization and text cleanup,
- semantic chunking,
- embedding generation using an embedding model (e.g., text-embedding-ada-002), and
- insertion and re-indexing in the Vector Database (FAISS or Chroma).

This process ensures that only human-verified, high-quality data populates the vector store, forming the factual foundation for the system's RAG-based retrieval.

D. LLM Integration with LangChain and LangGraph

LangChain abstracts the complexity of connecting the LLM with external data and specialized tools, focusing on prompt chaining and retrieval management. Crucially, the system proposes using LangGraph to manage the workflow within the Language Understanding Layer. LangGraph imposes a graph-based control flow, enabling the LLM to function as a "brain" that delegates sub-tasks and implements complex, non-linear reasoning steps required for adaptive conversational recommendation flows. This strategy ensures modularity and interpretability in dynamic recommendation scenarios.

E. Recommendation Engine Design

The core engine integrates several optimization techniques to enhance performance:

Retrieval-Augmented Generation (RAG):

RAG grounds LLM outputs in factual data by retrieving top-k relevant chunks from the Vector Store using cosine similarity. This eliminates hallucinations and allows the LLM to cite source-aligned results.

Fine-Tuning Mechanism:

Fine-tuning (e.g., QLoRA) adapts the base LLM to domain-specific terminology such as chipset names, battery types, camera modules, and display technologies.

Prompt Engineering Module:

Prompt Engineering ensures the system produces structured, concise, and context-aware outputs.

Techniques include:

Role prompting (e.g., "act as a tech advisor")

Schema-controlled prompting

Chain-of-Thought reasoning to enhance decision breakdowns

KNN with Cosine Similarity:

KNN is used to compute product similarity rankings.By comparing the angular distance between embedding vectors, the system identifies semantically aligned devices based on user needs, features, or constraints.

F. Integration with n8n for Workflow Automation and Manual Review

n8n functions as the Automation Layer, coordinating all asynchronous tasks required for data freshness and system reliability. It periodically triggers Firecrawl to scrape new product data and stores the results in a Review Database, initiating a Human-in-the-Loop (HITL) Review Subprocess.

Human reviewers evaluate each scraped entry for accuracy, completeness, and consistency. Only entries explicitly approved during this stage proceed further, preventing noisy or incorrect data from contaminating the knowledge base and reducing downstream hallucination risks.

IJARSCT
International Journal of Advanced Research in Science, Communication and Technology
International Open-Access, Double-Blind, Peer-Reviewed, Refereed, Multidisciplinary Online Journal
ISSN: 2581-9429
Volume 5, Issue 4, November 2025
Impact Factor: 7.67

After approval, n8n automatically executes the remaining pipeline steps—normalization, chunking, embedding generation, and Vector Database re-indexing—ensuring the knowledge base remains continuously synchronized and verified for RAG retrieval.

IV. METHODOLOGY

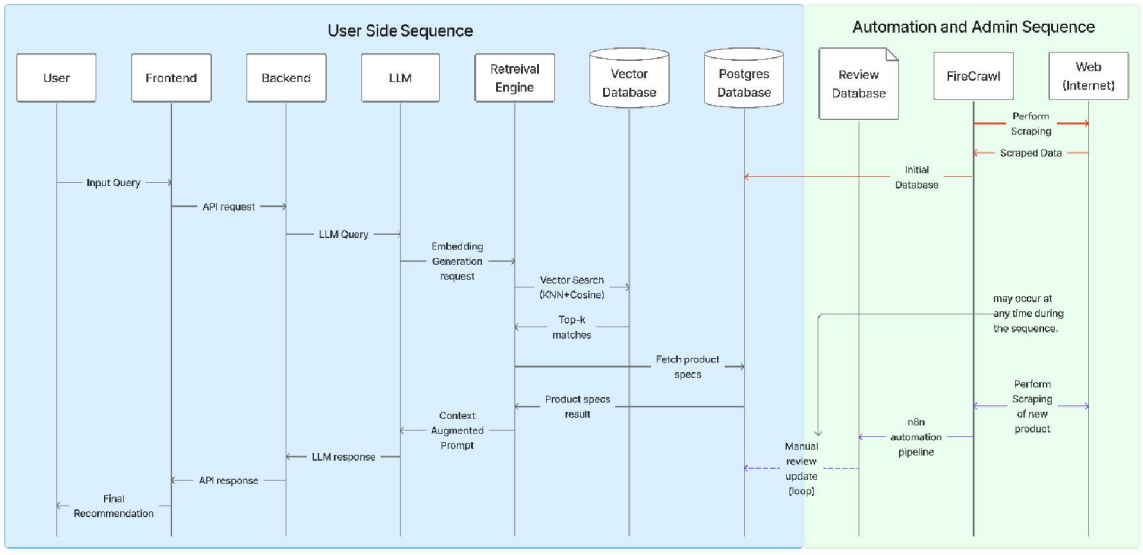
This section presents the conceptual methodology for the proposed LLM-Based Tech Product Recommendation System. The workflow integrates complementary AI techniques—Retrieval-Augmented Generation (RAG), Fine-Tuning, Prompt Engineering, and vector-based similarity computation—within an automated orchestration pipeline powered by LangChain, LangGraph, Firecrawl, and n8n. The methodology emphasizes data lifecycle management, LLM alignment, and adaptive reasoning to ensure accurate and context-aware recommendations.

A. Data Acquisition and Preparation

The methodology begins with establishing a continuous and reliable data pipeline to populate the proprietary knowledge base. The automation tool n8n periodically triggers Firecrawl to scrape updated product information such as specifications, features, and relevant descriptions from trusted public sources. Firecrawl converts raw webpage content into structured, machine-readable text.

A Human-in-the-Loop (HITL) process ensures data quality before indexing. Scraped entries are stored in a temporary Review Database, where human experts validate accuracy, consistency, and completeness. Only approved data proceeds to preprocessing, which includes normalization, text cleanup, and splitting the content into semantically meaningful chunks.

Each chunk is embedded into a high-dimensional vector representation using an embedding model such as text-embedding-ada-002. The resulting embeddings and their source text are stored in a Vector Database (e.g., FAISS or Chroma), forming the basis for semantic retrieval.



B. Retrieval-Augmented Generation (RAG) Workflow

RAG strengthens factual grounding by conditioning the LLM's output on retrieved external knowledge. The workflow consists of:

- Query Embedding – The user query is embedded using the same embedding model applied during indexing.

IJARSCT
International Journal of Advanced Research in Science, Communication and Technology
International Open-Access, Double-Blind, Peer-Reviewed, Refereed, Multidisciplinary Online Journal
ISSN: 2581-9429
Volume 5, Issue 4, November 2025
Impact Factor: 7.67

- Semantic Retrieval – A cosine-similarity search is executed on the Vector Store to retrieve the top-k most relevant product chunks.
- Prompt Augmentation – Retrieved text is inserted into a prompt template managed by LangChain, constraining the LLM to generate responses grounded only in the provided factual context.

This mechanism mitigates hallucination and ensures the recommendations remain up-to-date and verifiable.

C. LLM Fine-Tuning Strategy

To improve domain alignment, the methodology incorporates optional fine-tuning. Using approaches such as QLoRA, a pretrained LLM is adapted to tech-specific terminology, feature descriptions, and recommendation styles. QLoRA enables memory-efficient training through low-rank adapters applied to a quantized base model.

Fine-tuning enhances understanding of device specifications, reduces ambiguity, and improves the precision of final recommendations—complementing RAG's factual grounding with domain-aware reasoning.

D. Prompt Engineering Framework

Prompt Engineering serves as the runtime layer for constraining and structuring the LLM's output. Prompts embed explicit instructions regarding:

- task definition,
- required output format (e.g., comparison tables, ranked lists),
- constraints such as budget or use case,
- and integration of retrieved RAG context.

Techniques such as Role Prompting ("Act as a tech expert") and Chain-of-Thought (CoT) prompting guide the LLM through intermediate reasoning steps, improving clarity, interpretability, and the relevance of generated recommendations.

E. Product Similarity via KNN and Cosine Similarity

To support similarity-based recommendations, the methodology employs K-Nearest Neighbors (KNN) within the embedding space. Cosine Similarity quantifies how closely product vectors align with the user's query or preference embedding. Higher similarity scores indicate stronger semantic relevance.

KNN uses these metrics to identify the top matching products, which are integrated into the final recommendation pipeline alongside RAG-retrieved context.

F. Orchestration Using LangChain, LangGraph, and n8n

The entire recommendation workflow is orchestrated by a combination of LangChain, LangGraph, and n8n:

- LangChainmanages prompt construction, retrieval coordination, and tool invocation, forming the backbone of the LLM pipeline.
- LangGraphintroduces graph-based control flow, enabling non-linear, stateful reasoning paths. The Model Context Protocol (MCP) can complement these frameworks by standardizing how external tools and APIs are accessed by the LLM, reducing integration overhead.It allows the LLM to dynamically route tasks such as retrieval, similarity computation, and summary generation based on user intent.
- n8n maintains long-term operational integrity by automating scraping schedules, managing the human-review queue, and triggering preprocessing, embedding generation, and vector database re-indexing.

This hybrid offline–online methodology enables a continuously synchronized, human-verified knowledge base and an adaptive LLM response pipeline.

IJARSCT
International Journal of Advanced Research in Science, Communication and Technology
International Open-Access, Double-Blind, Peer-Reviewed, Refereed, Multidisciplinary Online Journal
ISSN: 2581-9429 Volume 5, Issue 4, November 2025 Impact Factor: 7.67

V. TECHNICAL CONSIDERATIONS AND EVALUATION METRICS

The hybrid LLM-based recommendation system introduces several non-functional constraints related to accuracy, latency, data freshness, and orchestration complexity. This section outlines the key technical considerations and defines the conceptual metrics used to assess system performance.

A. Technical Considerations and Constraints

- Scalability and Computational Efficiency: The combination of RAG, Fine-Tuning, and vector retrieval requires balancing accuracy with resource cost. Fine-tuning is the most computationally expensive component, even with techniques such as QLoRA. In contrast, Prompt Engineering and KNN similarity impose minimal overhead. As the knowledge base grows, the vector store must be periodically re-indexed or pruned to maintain retrieval performance.
- Latency and Response Time: Real-time recommendation systems require low response times. However, the system includes multiple sequential steps—query embedding, retrieval, LLM inference, and similarity scoring—each contributing to latency. Although RAG is faster than processing large context windows, multi-turn flows in LangGraph may increase delay if not optimized.
- Data Freshness and Factual Consistency: Tech product data changes rapidly, making continuous updates essential. The HITL workflow ensures high-quality data, but human review introduces a natural bottleneck. RAG helps mitigate staleness by grounding the model on updated evidence, but outdated embeddings, unreviewed entries, or inconsistent chunking may still degrade accuracy.
- Embedding and Retrieval Limitations: The system depends heavily on embedding quality. Generic embedding models may struggle to distinguish between highly similar devices or capture domain-specific attributes. Retrieval errors propagate directly to RAG output, affecting final recommendation accuracy.
- Orchestration Complexity and Integration Overhead: Coordinating LangChain, LangGraph, Firecrawl, n8n, and the Vector Database increases system complexity. Failure in any component can disrupt the pipeline, and graph-based workflows must be designed carefully to avoid unnecessary branching or redundant retrieval cycles.

B. Conceptual Evaluation Metrics

Recommendation Accuracy and RankingMetrics

Standard metrics from recommendation literature are used to assess ranking quality:

- Precision@K, Recall@K
- Hit Rate (HR@K)
- NDCG
- F1-score

These measure the relevance and order of recommended products.

Semantic and Contextual Alignment Metrics

Because the system relies on RAG and embeddings, semantic alignment is essential:

- Cosine Similarity (vector alignment)
- BERTScore (contextual text similarity)
- Faithfulness and context relevance (RAG consistency)

These metrics verify that outputs remain grounded in retrieved evidence.

Qualitative and Operational Metrics

User-facing and system reliability factors include:

- Latency
- Response relevance
- User satisfaction
- Coherence and conciseness

IJARSCT
International Journal of Advanced Research in Science, Communication and Technology
International Open-Access, Double-Blind, Peer-Reviewed, Refereed, Multidisciplinary Online Journal
ISSN: 2581-9429
Volume 5, Issue 4, November 2025
Impact Factor: 7.67

- Retry Rate
- Mean Conversation Length (MCL)

These metrics ensure the system is efficient, interpretable, and conversationally stable.

VI. EXPECTED OUTCOMES AND FUTURE WORKS

This section discusses the anticipated theoretical benefits of the proposed LLM-based recommendation system design and outlines prospective avenues for future research and conceptual system extensions. The design is expected to deliver significant theoretical benefits in accuracy, efficiency, and modularity.

A. Expected Outcomes and Theoretical Advantages

The proposed design, integrating RAG, Fine-Tuning, and comprehensive automation, is expected to yield significant improvements over traditional and generic LLM-based recommendation solutions:

- Enhanced Factual Accuracy and Explainability: The primary expected outcome is the mitigation of LLM hallucination through RAG, which grounds recommendations in retrieved, factual product data. RAG provides High Explainability by allowing the system to conceptually cite the sources or text chunks used to generate the answer. Furthermore, Chain-of-Thought (CoT) Prompting is expected to enhance LLM reasoning, leading to clearer explanations for comparisons.
- Automation and Modular Architecture: Continuous data synchronization is achieved via the n8n automation layer and Firecrawl, which addresses knowledge cut-off. The integration of LangChain for orchestration and LangGraph for graph-based control flow provides a modular, scalable architecture where the LLM acts as an adaptive "brain," managing complex, interactive, and multi-turn recommendation sequences.

B. Anticipated Challenges and Limitations

The conceptual model acknowledges several challenges inherent to integrating advanced LLM techniques:

- Computational Cost and Scalability: The primary challenge lies in computational requirements. Fine-Tuning remains the most resource-intensive method, even with efficient techniques like QLoRA, contributing to high overhead for large-scale serving.
- Knowledge Maintenance Trade-offs: While RAG is superior for handling Knowledge Volatility by allowing easy knowledge base updates, Fine-Tuning requires resource-intensive retraining if foundational domain knowledge or required output styles change significantly.
- Context Limitations: System quality is limited by the integrity of retrieved text chunks and the constraint imposed by the LLM's finite context window.

C. Future Enhancements and Research Directions

The conceptual design establishes a robust platform for numerous subsequent research directions:

- Voice-Enabled Interaction: A planned extension involves integrating a voice assistant that converts user speech into text prompts for the LLM. This will enable hands-free interaction, allowing users to request recommendations or comparisons using natural speech. The speech-to-text component can be tuned for regional accents and multilingual Indian users to ensure accurate query understanding.
- Market Adaptation and Global Scalability: Initially, the system will target the Indian technology market, using localized product data, pricing, and consumer trends to ensure relevance. After successful regional deployment, it can be extended to global markets by integrating international data sources and fine-tuning models for cross-market terminology and currency adaptation, maintaining contextual precision at scale.
- Advanced Personalization: Conceptual exploration can integrate Reinforcement Learning (RL) techniques to facilitate adaptive learning from user feedback, thereby optimizing long-term user satisfaction.Integration of the Model Context Protocol (MCP) can also be explored to support standardized, scalable tool interoperability for future system extensions.

IJARSCT
International Journal of Advanced Research in Science, Communication and Technology
International Open-Access, Double-Blind, Peer-Reviewed, Refereed, Multidisciplinary Online Journal
ISSN: 2581-9429
Volume 5, Issue 4, November 2025
Impact Factor: 7.67

VII. CONCLUSION

Hybrid, context-aware recommendation tasks increasingly demand architectures that combine factual grounding, adaptive reasoning, and continuous knowledge updates. This paper presented the conceptual design of an LLM-based tech product recommendation system addressing key limitations of traditional and ungrounded LLM pipelines, including hallucination, knowledge cut-off, and data sparsity. The proposed architecture integrates Retrieval-Augmented Generation (RAG), Fine-Tuning, and Prompt Engineering, supported by modular orchestration through LangChain and LangGraph, with data freshness maintained via an automated Firecrawl–n8n pipeline and Human-in-the-Loop (HITL) validation; semantic similarity is achieved through KNN with cosine similarity. Together, these components form a coherent, adaptive framework capable of producing accurate, explainable, and domain-aware recommendations. Future extensions include voice-enabled interaction for multilingual Indian users, global market adaptation through international data integration, and advanced personalization using Reinforcement Learning (RL), positioning the system as a scalable foundation for next-generation, user-centric recommendation technologies.

REFERENCES

[1] R. Pradhan, "RAG vs. Fine-Tuning vs. Prompt Engineering: A Comparative Analysis for Optimizing AI Models", IJCTEC, Volume 8, Issue 5, 2025

[2] S. Luo, Y. Yao, B. He, Y. Huang, A. Zhou, X. Zhang, Y. Xiao, M. Zhan, and L. Song, "Integrating large language models into recommendation via mutual augmentation and adaptive aggregation," arXiv preprint arXiv:2401.13870, 2024.

[3] S. Raza, M. Rahman, S. Kamawal, A. Toroghi, A. Raval, F. Navah, and A. Kazemeini, "A comprehensive review of recommender systems: Transitioning from theory to practice," arXiv preprint arXiv:2407.13699, 2024.

[4] E. Masciari, A. Umair, and M. H. Ullah, "A systematic literature review on AI-based recommendation systems and their ethical considerations," IEEE Access, vol. 12, pp. 121223–121239, 2024, doi: 10.1109/ACCESS.2024.3451054.

[5] S. Habil, S. El-Deeb, and N. El-Bassiouny, "AI-based recommendation systems: The ultimate solution for market prediction and targeting,"in Proc. Int. Conf. (Springer), 2023, pp. 1–15, doi: 10.1007/978-3-031-14961-0_30.

[6] M. O. Ayemowa, R. Ibrahim, and M. M. Khan, "Analysis of recommender system using generative artificial intelligence: A systematic literature review," IEEE Access, vol. 12, pp. 87742–87759, 2024, doi: 10.1109/ACCESS.2024.3416962.

[7] S. Carole, A. Poupi, and H.-C. Kim, "Enhanced experiences: Benefits of AI-powered recommendation systems," in Proc. Int. Conf. Adv. Commun. Technol. (ICACT), 2024, doi: 10.23919/ICACT60172.2024.10471918.

[8] D. Jannach, A. Manzoor, W. Cai, and L. Chen, "A survey on conversational recommender systems," ACM Comput. Surv., vol. 54, no. 4, art. 105, pp. 1–35, 2021.

[9] K. G. Yager, "Domain-specific chatbots for science using embeddings," Digital Discovery, vol. 2, no. 6, pp. 1850–1861, 2023.

[10] R. Zhang et al., "A self-enhancement approach for domain-specific chatbot training via knowledge mining and digest," arXiv preprint arXiv:2311.10614, 2023.

[11] R. Di Pasquale and S. Represa, "Empowering domain-specific language models with graph-oriented databases: A paradigm shift in performance and model maintenance," arXiv preprint arXiv:2410.03867, 2024.

[12] N. Vedula, M. Collins, E. Agichtein, and O. Rokhlenko, "Generating explainable product comparisons for online shopping," in Proc. ACM Int. Conf. Web Search Data Mining (WSDM), 2023, pp. 1–9, doi: 10.1145/3539597.3570489.

[13]J. Boegershausen, H. Datta, A. Borah, and A. T. Stephen,"Fields of gold: Scraping web data for marketing insights,"Journal of Marketing, vol. 86, no. 5, pp. 1–20, 2022.

[14] V. Carle,"Web scraping using machine learning,"M.S. thesis, School of Electrical Engineering and Computer Science, KTH Royal Institute of Technology, Stockholm, Sweden, 2020.

[15] Al-Hasan, T. M., Sayed, A. N., Bensaali, F., Himeur, Y., Varlamis, I., and Dimitrakopoulos, G., "From Traditional Recommender Systems to GPT-Based Chatbots: A Survey of Recent Developments and Future Directions," *Big Data and Cognitive Computing*, vol. 8, no. 36, 2024.

[16] Bodduluri, K. C., Palma, F., Kurti, A., Jusufi, I., and Löwenadler, H., "Exploring the Landscape of Hybrid Recommendation Systems in E-Commerce: A Systematic Literature Review," *IEEE Access*, vol. 12, pp. 28273–28294, 2024.
[17] Shafiee, S., "Unveiling the Latest Trends and Advancements in Machine Learning Algorithms for Recommender Systems: A Literature Review," *Procedia CIRP*, vol. 121, pp. 115–120, 2024.
[18] Meyer, S., Singh, S., Tam, B., Ton, C., and Ren, A., "A Comparison of LLM Fine-Tuning Methods and Evaluation Metrics with Travel Chatbot Use Case," *arXiv preprint arXiv:2408.03562*, 2024.
[19] Saindane, R. V., Patil, A. S., Bhoi, R. S., and Bhoite, S. Y., "Product and Price Comparison Using Machine Learning," *International Research Journal of Modernization in Engineering, Technology and Science*, vol. 6, no. 5, pp. 199–205, 2024.
[20] Chowdhury, R. D., Sarkar, A., Banik, M., and Bobbili, P. R., "Product Attribute Extraction and Product Listing Analysis from E-Commerce Websites," *ResearchGate Preprint*, 2023.
[21] Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L., "QLoRA: Efficient Finetuning of Quantized LLMs," *arXiv preprint arXiv:2305.14314*, 2023.
[22] Reddy, N. T. K., Patra, M. R., and Mishra, B. K., "Design and Implementation of an AI-Based Chatbot Framework with Retrieval-Augmented Generation and Integrated Recommender System," *SSRN*, 2025.
[23] Pokhrel, S., K. C., B., and Shah, P. B., "A Practical Application of Retrieval-Augmented Generation for Website-Based Chatbots: Combining Web Scraping, Vectorization, and Semantic Search," *Journal of Trends in Computer Science and Smart Technology*, vol. 6, no. 4, pp. 424–442, 2024.
[24] Huang, X., Lian, J., Lei, Y., Yao, J., Lian, D., and Xie, X., "Recommender AI Agent: Integrating Large Language Models for Interactive Recommendations," *arXiv preprint arXiv:2308.16505*, 2024.
[25] A. Blount, A. Gulli, S. Saboo, M. Zimmermann, and V. Vuskovic,"Introduction to agents and agent architectures,"Google& Kaggle Whitepaper, Nov. 2025. [Online]. Available: https://www.kaggle.com/whitepaper-agents
[26] M. Styer, K. Patlolla, M. Mohan, and S. Diaz,"Agent tools and interoperability with MCP,"Google& Kaggle Whitepaper, Nov. 2025. [Online]. Available: https://www.kaggle.com/whitepaper-agent-tools
[27] K. Milam and A. Gulli,"Context engineering: Sessions and memory,"Google & Kaggle Whitepaper, Nov. 2025. [Online]. Available: https://www.kaggle.com/whitepaper-context-engineering
[28] M. Subasioglu, T. Bulmus, and W. Bakkali,"Agent quality,"Google& Kaggle Whitepaper, Nov. 2025. [Online]. Available: https://www.kaggle.com/whitepaper-agent-quality
[29] LangChain Docs, "Docs by LangChain — The platform for agent engineering," LangChain, 2025. [Online]. Available: https://docs.langchain.com/Accessed: Nov. 14, 2025.
[30] LangGraph Docs, "LangGraph sets the foundation for how we can build and scale AI workloads," LangChain, 2025. [Online]. Available: https://www.langchain.com/langgraphAccessed: Nov. 14, 2025
[31] Firecrawl Docs, "Introduction | Firecrawl — Turn entire websites into LLM-ready markdown," Firecrawl, 2025. [Online]. Available: https://docs.firecrawl.dev/introductionAccessed: Nov. 14, 2025.
[32] n8n Integrations – Firecrawl, "Integrate Firecrawl with hundreds of other apps and services using n8n," n8n, 2025. [Online]. Available: https://n8n.io/integrations/firecrawl/Accessed: Nov. 14, 2025.
[33] FAISS Documentation, "Welcome to FAISS — efficient similarity search and clustering of dense vectors," Facebook AI Research, 2025. [Online]. Available: https://faiss.ai/index.html. Accessed: Nov. 14, 2025.
[34] IBM Technology, "Prompt Engineering for LLMs, PDL, &LangChain in Action," YouTube video, Nov. 10, 2025. [Online]. Available: https://youtu.be/cgVppD6paYo. Accessed: Nov. 14, 2025.
[35] IBM Technology, "MCP vs API: Simplifying AI Agent Integration with External Data," YouTube video, May. 5, 2025. [Online]. Available: https://youtu.be/7j1t3UZA1TY. Accessed: Nov. 14, 2025.
[36] IBM Technology, "RAG vs Fine-Tuning vs Prompt Engineering: Optimizing AI Models," YouTube video, Apr. 14, 2025. [Online]. Available: https://youtu.be/zYGDpG-pTho. Accessed: Nov. 14, 2025.
[37] IBM Technology, "What are Word Embeddings?," YouTube video, Aug. 20, 2024. [Online]. Available: https://youtu.be/wgfSDrqYMJ4. Accessed: Nov. 14, 2025.

IJARSCT
International Journal of Advanced Research in Science, Communication and Technology
International Open-Access, Double-Blind, Peer-Reviewed, Refereed, Multidisciplinary Online Journal
ISSN: 2581-9429 Volume 5, Issue 4, November 2025 Impact Factor: 7.67

[38] IBM Technology, "What is MCP? Integrate AI Agents with Databases & APIs," YouTube video, Feb. 19, 2025. [Online]. Available: https://youtu.be/eur8dUO9mvE. Accessed: Nov. 14, 2025.
[39] IBM Technology, "What is a Vector Database? Powering Semantic Search & AI Applications," YouTube video, Mar. 14, 2025. [Online]. Available: https://youtu.be/gl1r1XV0SLw. Accessed: Nov. 14, 2025.